

SECTION 6 — GENERICS (Senior SDET Masterclass)

Topics Covered

- 6.1 Why Generics (Compile-Time Type Safety & Eliminating Boilerplate Casts)
- 6.2 Generic Classes (Parameterized Framework Types & Type Boundaries)
- 6.3 Generic Methods (Type Inference, Scoped Type Parameters, & Static Helpers)
- 6.4 Bounded Type Parameters (`<T extends SuperType>` Constraints)
- 6.5 Wildcards (`<?>` , `<? extends T>` , and `<? super T>`)
- **6.6 The PECS Principle (*Producer extends* , *Consumer super*)**
- 6.7 Type Erasure Mechanics (Compile-Time Desugaring & Bytecode Traces)
- 6.8 Synthetic Bridge Methods (Preserving Polymorphic Dispatch Post-Erasure)
- 6.9 Generic Restrictions & JVM Traps (`new T()` , Arrays, Primitives, Static Contexts)
- 6.10 Generics in Automation Frameworks (Fluent POM & Type-Safe API Clients)
- 6.11 High-Stakes Senior Generics Interview Questions & Spoken Solutions

6.1 Why Generics & 6.2 Generic Classes

1. Theory & Core Mechanics

Introduced in Java 5, Generics provide **Compile-Time Type Safety**:

- **Before Generics (Java 1.4 and earlier):** Collections operated exclusively on raw `java.lang.Object` . Developers had to manually cast objects upon retrieval (`(String) list.get(0)`). If an incorrect type was inserted, failures manifested at runtime via `java.lang.ClassCastException` .
- **With Generics:** The Java compiler validates that only compatible types are inserted. It automatically inserts the cast instructions during compilation, guaranteeing that runtime code is completely free of `ClassCastException` for properly typed collections.

```
// Non-Generic Legacy Collection (Dangerous)
List rawList = new ArrayList();
rawList.add("Deloitte");
rawList.add(100); // Compiles without error
String val = (String) rawList.get(1); // RUNTIME CRASH: ClassCastException!

// Generic Collection (Safe)
List<String> typedList = new ArrayList<>();
typedList.add("Deloitte");
// typedList.add(100); // COMPILE ERROR: Incompatible types
```

6.3 Generic Methods & 6.4 Bounded Type Parameters

1. Theory & Scoped Type Parameters

A generic method introduces its own type parameter independent of whether the enclosing class is generic:

- The type parameter is declared before the return type: `public static <T> T execute(...)` .
- **Bounded Type Parameters:** Constrain the valid types using the `extends` keyword (`<T extends Number>` , `<T extends BasePage & Validatable>`).

```
package com.deloitte.sdet.utils;

import java.util.List;

public final class CollectionUtils {

    // Generic static utility method with type inference
    public static <T> T getFirstOrThrow(List<T> items, String errorMessage) {
        if (items == null || items.isEmpty()) {
            throw new IllegalArgumentException(errorMessage);
        }
        return items.get(0); // Compiler infers exact return type T
    }

    // Bounded type: T must be a subclass of Comparable
    public static <T extends Comparable<T>> T findMax(T a, T b) {
        return a.compareTo(b) >= 0 ? a : b;
    }
}
```

6.5 Wildcards & 6.6 The PECS Principle

COVARIANCE (Read-Only)

<code><? extends T></code>
PRODUCER: Supplies data
Safe to READ as `T`
CANNOT write (except null)

CONTRAVARIANCE (Write-Only)

<code><? super T></code>
CONSUMER: Receives data
Safe to WRITE `T`
Reads return raw `Object`

The PECS Rule: Producer `extends` , Consumer `super`

- Use `<? extends T>` when your collection **PRODUCES** data to your method:
You are only reading elements out of the collection. You can read them safely as type `T` . You CANNOT insert anything (except `null`) because the compiler does not know the specific concrete subtype at runtime.
- Use `<? super T>` when your collection **CONSUMES** data from your method:
You are adding elements of type `T` into the collection. You can safely insert `T` or any subclass of `T` . When reading from it, elements are only guaranteed to be of type `Object` .

Production Code: The Canonical PECS Example

```

package com.deloitte.sdet.pecs;

import java.util.List;

public class PecsDemonstration {

    // PRODUCER: Reads items out of src (Producer extends)
    // CONSUMER: Adds items into dest (Consumer super)
    public static <T> void copy(List<? extends T> src, List<? super T> dest) {
        for (T item : src) {
            dest.add(item); // Safe: dest accepts T or supertypes of T
        }
    }
}

```

6.7 Type Erasure Mechanics & 6.8 Synthetic Bridge Methods

1. Theory & Bytecode Desugaring

Generics are a **compile-time syntactic illusion**. To ensure 100% backward compatibility with pre-Java 5 legacy bytecode, the Java compiler executes **Type Erasure**:

1. All generic type parameters are replaced with their **bound** (e.g. `T extends Number` becomes `Number`) or `java.lang.Object` if unbounded.
2. The compiler inserts explicit cast instructions in the bytecode where appropriate.
3. At runtime, the JVM has **zero knowledge** of whether an `ArrayList` was declared as `ArrayList<String>` or `ArrayList<Integer>`; both are simply raw `ArrayList`.

```

// Source Code:
public class Holder<T> {
    private T value;
    public void set(T val) { this.value = val; }
    public T get() { return value; }
}

// Bytecode Representation after Type Erasure:
public class Holder {
    private Object value;
    public void set(Object val) { this.value = val; }
    public Object get() { return value; }
}

```

2. Synthetic Bridge Methods

When a class extends a generic class or implements a generic interface and specifies a concrete type, type erasure creates a method signature mismatch:

```

public class StringHolder extends Holder<String> {
    @Override

```

```
public void set(String val) { super.set(val); }
}
```

After erasure, `Holder.set(Object)` exists, but `StringHolder` declared `set(String)`. To preserve dynamic method dispatch (`invokevirtual`), the compiler automatically synthesizes a **hidden Bridge Method**:

```
// Compiler-generated Synthetic Bridge Method in StringHolder.class:
public void set(Object val) {
    this.set((String) val); // Casts and delegates to set(String)
}
```

6.9 Generic Restrictions & JVM Traps

Because of Type Erasure, several intuitive operations are **strictly illegal** in Java:

1. Cannot Instantiate Type Parameters Directly (`new T()`):

- `T obj = new T();` \$to\$ **Compile Error**. At runtime, the JVM does not know what constructor to call because `T` is erased to `Object`.
- *Senior Fix*: Pass the `Class<T>` token: `clazz.getDeclaredConstructor().newInstance()`.

2. Cannot Create Generic Arrays (`new T[10]`):

- `T[] array = new T[10];` \$to\$ **Compile Error**. Java arrays are **reifiable** (they retain their component type at runtime), whereas generics are erased. A generic array would lead to heap pollution.

3. Cannot Use Primitive Types in Generics:

- `List<int>` \$to\$ **Compile Error**. Type erasure maps to `Object`, and primitives do not inherit from `java.lang.Object`. Must use boxed wrappers (`List<Integer>`).

4. Cannot Reference Type Parameters in **static** Context:

- `private static T sharedState;` \$to\$ **Compile Error**. Static fields belong to the class, not to any parameterized instance.

5. Cannot Perform **instanceof** on Parameterized Types:

- `if (list instanceof List<String>)` \$to\$ **Compile Error**. Because of erasure, the JVM only knows it is a `List`. You can only check `if (list instanceof List<?>)`.

6.10 Generics in Automation Frameworks

1. Fluent Generic Page Object Model

Enables type-safe page transitions without casting:

```
package com.deloitte.sdet.pages;

import org.openqa.selenium.WebDriver;

public abstract class BasePage<T> extends BasePage<T>> {

    protected final WebDriver driver;
```

```

protected BasePage(WebDriver driver) {
    this.driver = driver;
}

// Type-safe self-referential generic return for fluent chaining
@SuppressWarnings("unchecked")
public T refreshPage() {
    driver.navigate().refresh();
    return (T) this;
}

}

public class DashboardPage extends BasePage<DashboardPage> {
    public DashboardPage(WebDriver driver) { super(driver); }
    public DashboardPage filterByActive() { /* ... */ return this; }
}

```

2. Production-Grade Generic REST API Client

Demonstrates generic payload deserialization using `Class<T>` type tokens:

```

package com.deloitte.sdet.api;

import io.restassured.RestAssured;
import io.restassured.response.Response;

public final class GenericApiClient {

    private final String baseUrl;

    public GenericApiClient(String baseUrl) {
        this.baseUrl = baseUrl;
    }

    /**
     * Executes GET request and auto-deserializes JSON response into specified POJO type.
     */
    public <T> T get(String endpoint, Class<T> responseClass) {
        Response response = RestAssured.given()
            .baseUrl(baseUrl)
            .when()
            .get(endpoint)
            .then()
            .statusCode(200)
            .extract()
            .response();

        // Type-safe generic deserialization via Jackson
        return response.as(responseClass);
    }
}

```

6.11 High-Stakes Senior Generics Interview Questions & Spoken Solutions

Q1: "Explain PECS with a real-world scenario from test automation."

*"PECS stands for **Producer extends** , **Consumer super** . It dictates how to use wildcards when designing flexible API methods.*

*In our test automation framework, consider a test listener that aggregates test execution results. If we write a utility to merge test failure records:

```
public void processFailures(List<? extends FailureRecord> incomingFailures)
```

Here, `incomingFailures` is a **Producer**: our method only reads failure events out of the list to log them to our reporting database. By using `<? extends FailureRecord>` , the method can accept a `List<UiFailureRecord>` , `List<ApiFailureRecord>` , or `List<PerformanceFailureRecord>` .

*Conversely, if our method writes newly generated failure events into a shared sink:

```
public void appendFailures(List<? super FailureRecord> destinationSink)
```

Here, `destinationSink` is a **Consumer**: our method actively writes `FailureRecord` objects into it. Using `<? super FailureRecord>` allows the caller to pass a `List<FailureRecord>` or a `List<Object>` .

Following PECS ensures our framework utilities achieve maximum API flexibility without sacrificing compile-time type safety."

Q2: "Why does `List<String>` not inherit from `List<Object>` in Java?"

*"Because Java generics are **invariant**. If `List<String>` were a subtype of `List<Object>` (covariance), it would violate type safety:

```
List<String> stringList = new ArrayList<>();
List<Object> objectList = stringList; // If this were allowed...
objectList.add(100); // 100 is an Object, perfectly legal!
String s = stringList.get(0); // RUNTIME CRASH: Integer cannot be cast to String!
```

To prevent this runtime corruption, the compiler forbids assigning `List<String>` to `List<Object>` . If you need a method to accept any list regardless of its type parameter, you must use the unbounded wildcard `List<?>` ."

6.5 Enterprise Relevance at 5,000+ Test Scale

Generic Page Factory base classes (`BasePage<T>` extends `BasePage<T>>`) enable type-safe method chaining across multi-tenant test suites.